Explorador interactivo de ejercicios

UNIVERSIDAD TECNOLÓGICA METROPOLITANA

Tarea 3 | Proyecto web: catálogo de gimnasio Fuerza Base

Alumno: Saul Ulises Claudio Chan

Carrera: Desarrollo de Software Multiplataforma

Asignatura: Aplicaciones web

Docente: Michelle Alessandra Quintal Warner

Grupo: 4B

Fecha: 27 de octubre de 2026

Versión original sobre un catálogo de ejercicios de gimnasio. Completa las evidencias señaladas con información propia antes de entregar.

1. Propuesta

Fuerza Base ayuda a explorar ejercicios disponibles en un catálogo de fitness. Consulta datos reales desde la API REST de wger. Presenta nombre, músculos principales, equipo, descripción y una imagen cuando exista en la ficha. Los visitantes pueden buscar, filtrar, marcar favoritos y cambiar la presentación.

2. Requisitos y realización prevista

Requisito	Implementación
HTML, CSS y JavaScript	index.html, style.css y script.js en la raíz del repositorio.
API real con fetch()	GET a wger /api/v2/exerciseinfo/?limit=50.
10 elementos con imagen y descripción	La app solicita hasta 50 registros por página; imágenes/descripciones dependen de la ficha y su ausencia se indica.
Buscador y filtros	Búsqueda por texto, filtros de músculos y equipo en la página cargada.
Favoritos y localStorage	Guardar IDs en localStorage y activar vista de favoritos.
Errores y estados	Avisos de carga, problema de red, fallo de API y filtros sin coincidencias.
Diseño adaptable	Cuadrícula responsive, botones y campos etiquetados.
Publicación Pages	Configurar Pages en el repositorio y probar la URL generada.
Extra	Tema claro/oscuro y orden alfabético A–Z.

3. Archivos del repositorio

index.html contiene el encabezado, buscador, filtros y lista. **style.css** define la marca Fuerza Base y la vista responsive. **script.js** consume la API, interpreta sus resultados, filtra la página actual y conserva favoritos. **README.md** reúne fuente, atribución, límites, instrucciones y espacios para las evidencias.

4. Flujo de consulta

Al abrir la página, fetch() solicita una página de ejercicios y recibe JSON con resultados y enlaces de paginación. El código comprueba la respuesta antes de renderizar. El buscador y los filtros operan sobre la página descargada; los botones anterior/siguiente siguen los enlaces entregados por la API. Los favoritos permanecen en el navegador. Si falta foto o descripción, se informa esa condición.

5. Desarrollo solo desde GitHub

La consigna pide crear y modificar los archivos usando el editor web de GitHub. Copia cada archivo en el repositorio público llamado **proyecto-web-ia**. Conserva evidencia de seis commits que representen cambios reales, crea Issues para tareas auténticas, y abre un Pull Request para una mejora. Activa GitHub Pages desde la rama principal y carpeta raíz; luego revisa la página publicada.

Plan sugerido de commits

1) estructura HTML y README; 2) estilos responsive; 3) primer fetch y renderizado; 4) filtros y búsqueda; 5) favoritos y localStorage; 6) modo claro/oscuro y documentación. Haz cada commit al terminar un cambio real y escribe un mensaje que explique ese cambio.

README y demostración

Anota integrantes reales, qué hace el sitio, cómo usarlo, endpoint y atribución, límites de datos, tres prompts reales, respuesta de IA corregida, error que sí encontraste, reflexión, URL de Pages y URL del repositorio. En la exposición, cada integrante debe poder explicar fetch(), HTML/CSS/JavaScript, filtros, fallos, publicación y mejora extra.

6. Entregables y pruebas personales

Repositorio: [URL después de crearlo]
GitHub Pages: [URL después de publicarlo y verificarlo]
Issues: [enlaces reales]
Pull Request: [enlace real]
Capturas: [insertar capturas propias]

7. Preguntas de presentación

fetch() inicia la petición web y devuelve una promesa; el programa comprueba el estado y procesa el JSON.
HTML/CSS/JavaScript: contenido/estructura, presentación, interacción y lógica.
Buscador: compara texto con nombre/descripción de los registros cargados.
API no disponible: captura el error y muestra un mensaje.
Código de IA: identifica propuestas recibidas y explica correcciones reales.
GitHub Pages: servicio de publicación de archivos estáticos del repositorio, que debe habilitarse y verificarse.

Fuente de datos

La documentación de wger indica que su REST API vive bajo /api/v2/, ofrece endpoints públicos de ejercicios sin autenticación y pagina sus respuestas. Los ejercicios y las imágenes dependen del catálogo disponible; respeta la atribución/licencia indicada por cada registro. Referencia oficial: https://wger.readthedocs.io/en/latest/api/api.html